

Task 5 — Cinema Booking System (ASP.NET Core MVC)

User Stories

As a movie fan, I want to:

- Browse the movies that are currently showing without having to log in.
- See where and when each movie is playing.
- Create an account and book tickets for the showtime I want.
- See my past and upcoming bookings, and cancel a booking if my plans change.

As an admin, I want to:

- Add and update movies, including their posters.
- Manage the cinemas, the halls inside each cinema, and the showtimes for every movie.
- See all the bookings across the system.

Requirements

- Anyone can browse movies and showtimes without an account.
- Only logged-in users can book tickets and see their booking history.
- Only admins can manage movies, cinemas, halls, and showtimes.
- An admin account must be created on the first run of the application.
- Each booking must respect the seat capacity of the hall. A user cannot book more seats than what is left.
- A user cannot book a showtime that has already started.
- A user can cancel a booking only if the showtime has not started yet.
- Movie posters must be uploadable and must show on the public pages.
- The website must look acceptable on both desktop and mobile screens.
- The user must always see a clear success or error message after every action.

Tasks

1. Design the domain model and the database for the cinema system. Decide the entities, the relationships, the business rules, and the validation requirements.
2. Implement registration, login, logout, and change password using ASP.NET Core Identity.
3. Implement two roles: customer and admin. Make sure the admin account exists on first run.
4. Build the public pages: home page, movies list, movie details with showtimes, and a cinemas page.

5. Build the customer pages: book a ticket for a showtime, view my bookings, cancel a booking.
6. Build the admin section as a separate area: manage movies (with poster upload), categories, cinemas, halls, and showtimes.
7. Validate all user input on both the client side and the server side. Show errors next to the field that caused them.
8. Show the total ticket price live on the booking page as the user changes the number of seats, without reloading the page.
9. Show a success or error message after every create, update, or delete operation.
10. Make sure unauthorized users cannot see admin links and cannot reach admin pages directly through the URL.
11. Reject any booking that tries to take more seats than what is available in the hall.
12. Show friendly custom pages for 404 and 500 errors.
13. Make sure the website is usable on mobile screens.

Submission Rules

- A working .NET 10 MVC project in a public GitHub repo.
- README must explain how to run the project, how to log in with the default admin account, and where the uploaded posters are stored.
- Include the migration files in the repo.